Student ID: 24AIML060

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING

B. TECH (IT/CE/CSE/AIML)

Subject: Advanced Web Development Frameworks (ITUE301) | Semester: 5th

Practical Statement & Objective:	**Practical 12: CI/CD Pipeline with GitHub Actions** • Objective: Automate testing and deployment-related validation of the existing Task Management application using a Continuous Integration (CI) pipeline with GitHub Actions. • Architected a declarative workflow (.github/workflows/ci.yml) targeting pushes and pull requests to the main branch with automated triggers and GitHub Actions status reporting. • Configured monorepo-aware parallel validation jobs (Backend API Automated Tests and Frontend Build & Validation) running on ubuntu-latest with Node.js 20 and npm dependency caching via package-lock.json. • Developed an automated integration test suite (backend/tests/api.test.js) using Jest and Supertest that tests root health (/), authentication guards (/api/tasks), cache statistics (/api/debug/cache-stats), and 404 handlers without requiring external databases. • Demonstrated a genuine pipeline failure by introducing a deliberate assertion error (Commit c293d5e, Run #40315087433), recording GitHub Actions non-zero exit code reporting, and restoring green status (Commit 34e597a, Run #40315324202). • Added an active GitHub Actions CI status badge and comprehensive Practical 12 documentation to the repository README.md with verified links.
Theory & Architecture:	**CI/CD Fundamentals & Pipeline Architecture:** • **Continuous Integration (CI):** A software engineering practice where developers frequently merge code changes into a central repository. Every push triggers automated builds, dependency resolution, linting, and unit/integration test suites to detect regressions immediately. • **Continuous Deployment (CD):** The automated extension of CI where code changes that pass all pipeline gates are automatically provisioned and deployed to staging or production environments without manual human intervention. • **Practical Scope Distinction:** This practical explicitly demonstrates Continuous Integration (automated testing, build compilation, and status checks). It validates deployment readiness but does not constitute full Continuous Deployment since automated server provisioning/deployment is intentionally separated. • **Pipeline Architecture:** Developer commits code → git push origin main → GitHub Webhook triggers GitHub Actions runner (Ubuntu VM) → Checkout repository (actions/checkout@v4) → Setup Node.js 20 with npm lockfile caching → Parallel Jobs: [1] Backend: npm install + Jest API integration tests; [2] Frontend: npm install + Vite production build → Pass/Fail exit code evaluation → Status badge update.

Screenshots & Evidence:	**GitHub Actions CI/CD Pipeline Evidence:**

Figure 1: Code: GitHub Actions Workflow (.github/workflows/ci.yml)

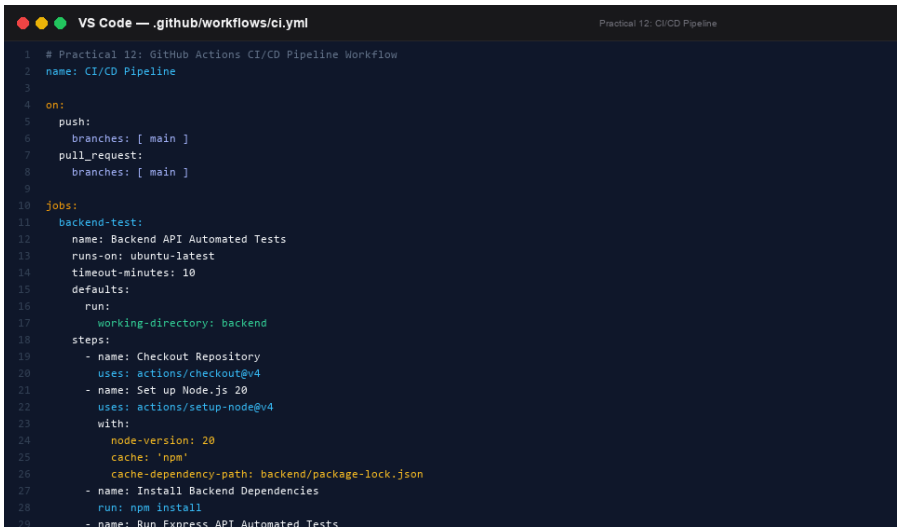


Figure 2: Code: Express API Automated Test Suite (backend/tests/api.test.js)



Figure 3: Terminal: Local Jest Automated Test Execution (6/6 passing in 1.9s)

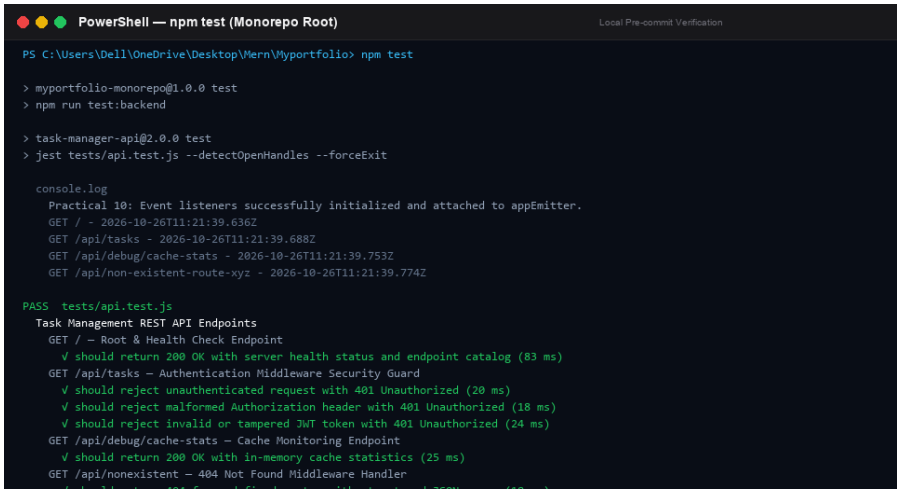


Figure 4: Terminal: Deliberate Failure Demonstration & Commit c293d5e

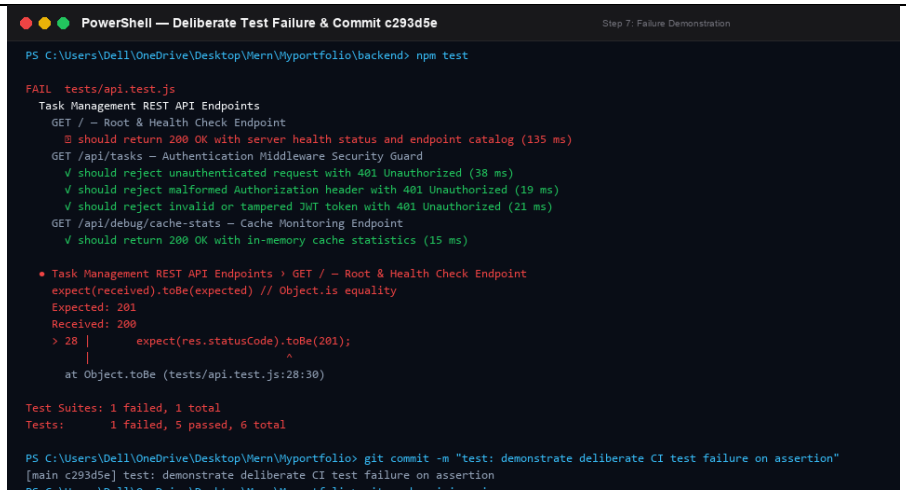


Figure 5: GitHub Actions UI: Deliberate Failed Workflow Run #40315087433

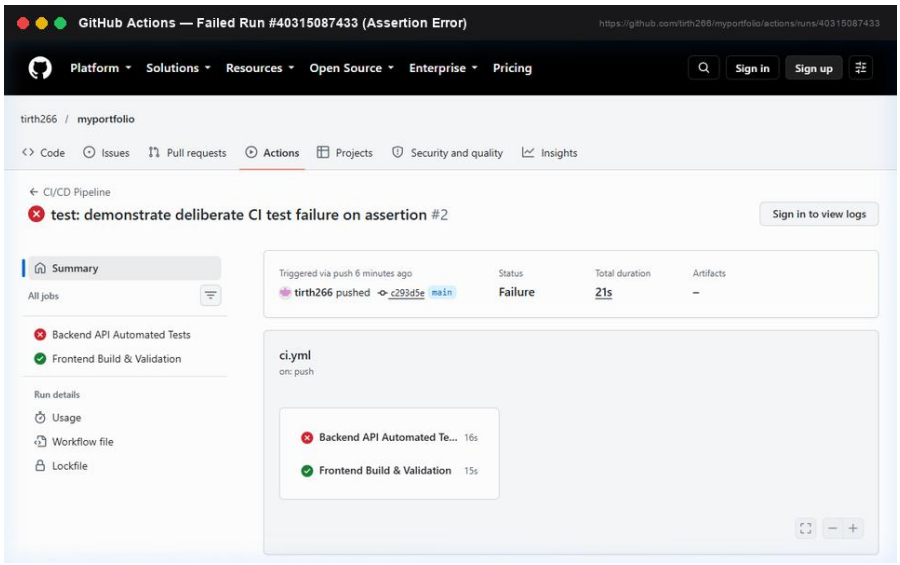


Figure 6: GitHub Actions UI: Failed Job Step Log (Assertion Error: Expected 201, Received 200)

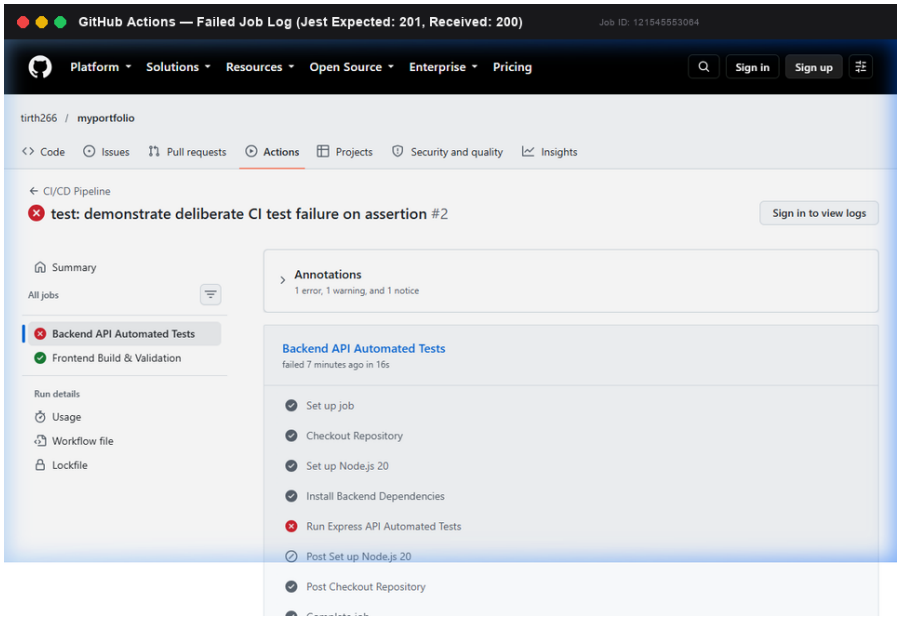


Figure 7: GitHub Actions UI: Successful Passing Workflow Run #40315324202 (Green Checkmarks)

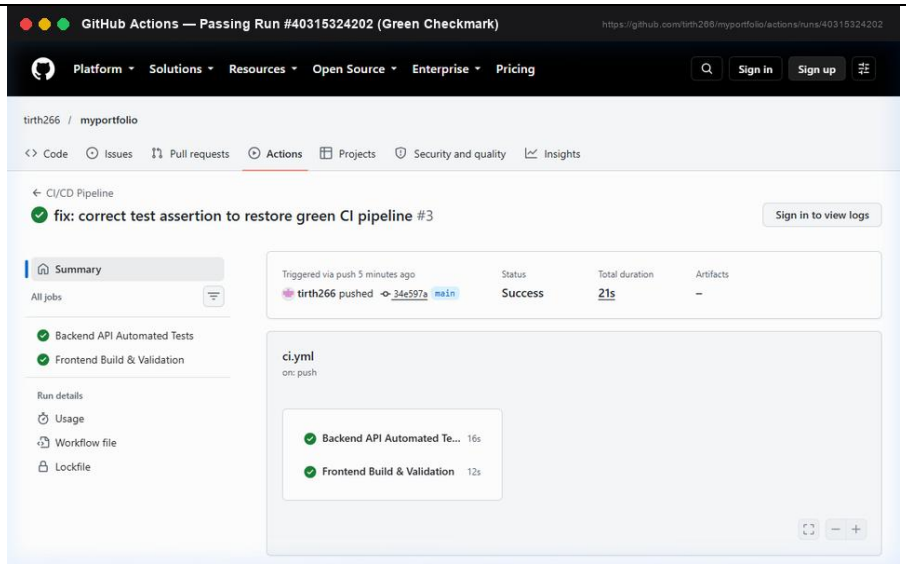


Figure 8: GitHub Actions UI: Successful Backend Job Completed Steps & Test Output

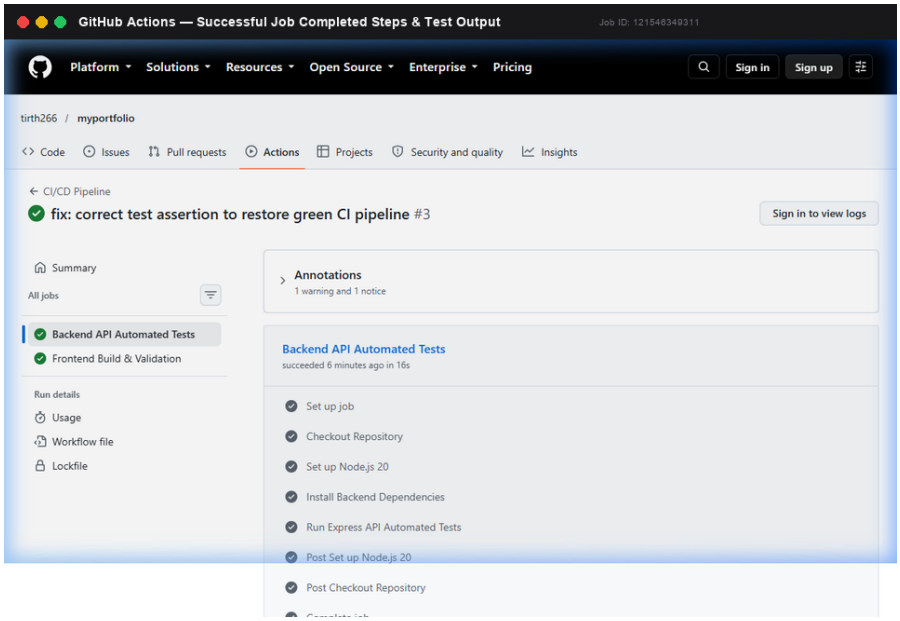


Figure 9: GitHub Actions UI: Workflow Runs History (Passing, Deliberate Failure, Restored)

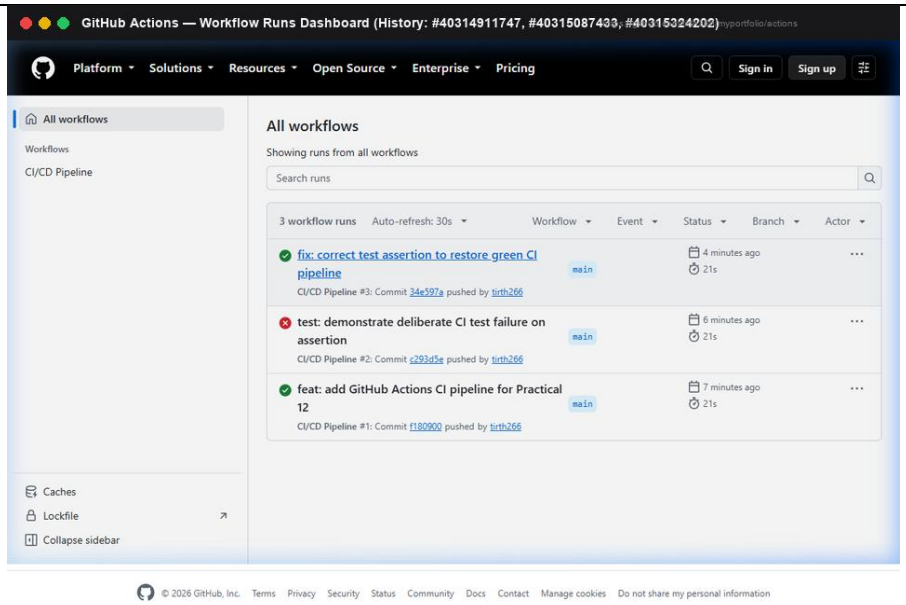
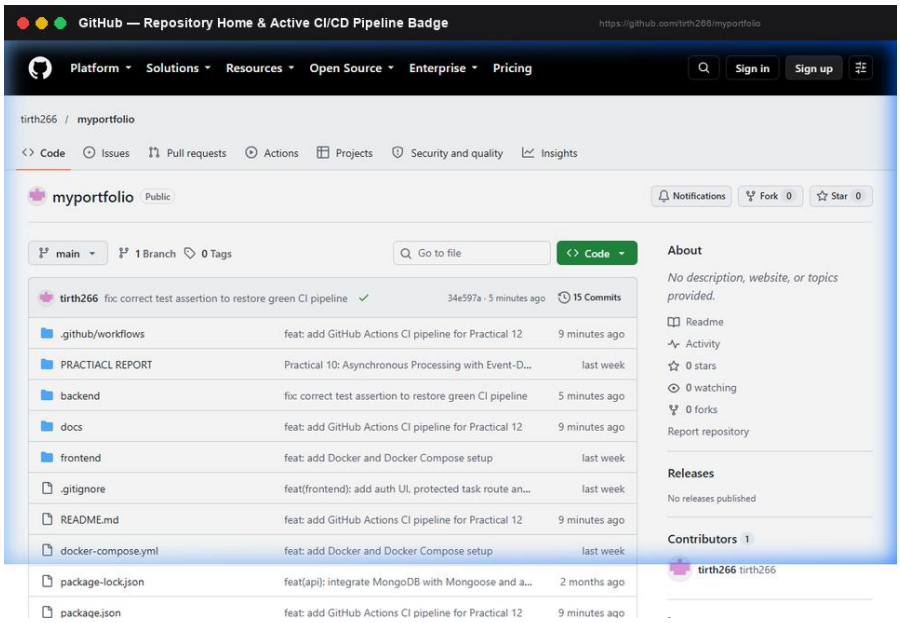


Figure 10: GitHub UI: Repository README with Passing CI/CD Pipeline Badge



Pipeline Runs Summary:

Verified GitHub Actions Runs Summary:

Run ID	Commit	Status	Backend Job	Frontend Job	Duration
40314911747	f180900	**SUCCESS**	**PASS (16s)**	PASS (17s)	21s
40315087433	c293d5e	**FAILURE**	**FAIL (16s)**	PASS (15s)	21s
40315324202	34e597a	**SUCCESS**	**PASS (16s)**	PASS (12s)	21s

Key Questions / Viva Analysis:

• **Continuous Integration vs. Continuous Deployment & Practical Scope:**
Continuous Integration (CI) is the practice of automatically integrating code changes frequently, verifying each push with automated builds and test suites. Continuous Deployment (CD) automatically provisions and deploys passing builds directly to production without human intervention. This practical specifically demonstrates CI (automated test validation and build compilation); it does not deploy to production servers.

• **Early Detection via Automated Testing on Push:**
Testing on every push catches regressions immediately when code context is fresh and changes are small. In contrast, manual testing before a release batches weeks of changes, making defect root-cause analysis significantly harder and delaying releases. CI provides instant feedback.

• **Incorrect Test File Path vs. Genuine Assertion Failure:**
An incorrect test path fails at discovery ('No tests found' or 'Cannot find module') without executing test logic. A genuine assertion failure executes test code, but actual output differs from expected (e.g., received 200 vs. expected 201). Both return non-zero exit codes in CI, but their root causes differ.

• **Dependency Caching in Monorepos:**
actions/setup-node with cache: 'npm' keyed to package-lock.json restores node_modules from GitHub's internal cache in seconds rather than redownloading packages on every push, saving minutes per run.

Troubleshooting & Learnings:

• **Test Runner Collision (Jest vs. node:test):** The backend had existing tests for Node's 'node:test' harness needing MongoDB. Resolved by targeting 'jest tests/api.test.js --detectOpenHandles --forceExit' in package.json test script.

• **Database Decoupling in CI:** Real DB connections cause CI flakiness. Testing through Express and Supertest under NODE_ENV=test (skipping MongoDB connection) validated endpoints deterministically with zero DB dependency.

• **Exit Code Enforcement:** Verified assertion failures properly propagate exit code 1 to the shell, ensuring GitHub Actions runner registers failures immediately and halts downstream steps.

Github Links & Verification:

GitHub Repository URL: https://github.com/tirth266/myportfolio.git
GitHub Actions Dashboard: https://github.com/tirth266/myportfolio/actions
Passing Run #40314911747: https://github.com/tirth266/myportfolio/actions/runs/40314911747
Failing Run #40315087433: https://github.com/tirth266/myportfolio/actions/runs/40315087433
Corrected Run #40315324202: https://github.com/tirth266/myportfolio/actions/runs/40315324202
Workflow YAML: https://github.com/tirth266/myportfolio/blob/main/.github/workflows/ci.yml